Capitolo 2

Java e programmazione orientata agli oggetti

2.1 Java: compilazione ed esecuzione

La *programmazione orientata agli oggetti*, o *OOP* (*Object-Oriented Programming*), organizza il programma in oggetti che collaborano. Un *oggetto* è un'entità software con uno stato, cioè dati memorizzati, e operazioni che agiscono su tale stato. Una *classe* ne definisce struttura e comportamento: ogni oggetto è un'*istanza* di una classe, mentre una classe può avere zero o più istanze durante l'esecuzione. Java offre un ambiente di compilazione ed esecuzione e costrutti con cui esprimere classi, interfacce, incapsulamento, ereditarietà e polimorfismo.

2.1.1 JDK, JRE, JVM, bytecode e JIT

Il *JDK* (*Java Development Kit*) è l'insieme completo degli strumenti necessari per sviluppare in Java. Include il compilatore `javac`, strumenti di supporto e la *JRE*. La *JRE* (*Java Runtime Environment*) è l'ambiente di esecuzione: comprende le librerie standard e la *JVM*. La *JVM* (*Java Virtual Machine*) è la macchina virtuale che esegue il codice intermedio prodotto dalla compilazione.

Il codice sorgente Java, contenuto in file `.java`, viene compilato da `javac` in *bytecode*, contenuto normalmente in file `.class`. Il bytecode è un formato intermedio: non è codice macchina specifico per una CPU, ma non è nemmeno codice sorgente Java; è pensato per una macchina astratta, la JVM. Durante l'esecuzione, la JVM interpreta o traduce il bytecode nelle istruzioni native dell'architettura. Il *JIT* (*Just In Time*) è il compilatore dinamico interno alla JVM che può trasformare parti di bytecode in codice macchina durante l'esecuzione.

Il flusso essenziale è:

$$\text{sorgente } .java \xrightarrow{\texttt{javac}} \text{bytecode } .class \xrightarrow{\text{JVM/JIT}} \text{istruzioni native} \xrightarrow{\text{CPU}} \text{esecuzione.} \tag{2.1}$$

Nella formula, `javac` indica il compilatore statico, JVM/JIT indica l'ambiente che esegue o compila dinamicamente il bytecode, e CPU indica il processore fisico che esegue le istruzioni native. Le operazioni di input/output (I/O) comprendono interazione con l'utente, file, connessioni di rete e socket, e usano librerie e risorse esterne durante l'esecuzione.

Ogni opcode del bytecode è codificato in un byte; un'istruzione può comprendere anche operandi. L'indipendenza dalla CPU riguarda il bytecode: la JVM deve essere disponibile per la piattaforma concreta.

2.1.2 Compilazione, avvio e metodo main

La compilazione di una classe avviene con il comando:

```
javac NomeClasse.java
```

che genera il file `NomeClasse.class`. L'esecuzione avviene con:

```
java NomeClasse
```

a condizione che la classe contenga il metodo di avvio:

```
public static void main(String[] args)
```

La parola chiave `static` indica che un campo o un metodo appartiene alla classe e non alla singola istanza. Poiché `main` è statico, al suo interno si possono chiamare direttamente altri metodi statici; per chiamare un metodo non statico occorre invece disporre di un'istanza.

Per convenzione, i nomi di classe iniziano con lettera maiuscola; i nomi di variabile e metodo usano il *camelCase* con iniziale minuscola; i metodi hanno nomi verbali, mentre le classi sono descritte preferibilmente da sostantivi. Questa distinzione è coerente con l'OOP: le classi modellano entità del dominio, mentre i metodi rappresentano azioni o servizi.

Da JDK 11 si può anche avviare direttamente un sorgente con `java NomeClasse.java`: il launcher gestisce la compilazione, senza richiedere di produrre manualmente il file `.class`. Le due modalità hanno argomenti diversi: `java NomeClasse` indica il nome della classe compilata, mentre `java NomeClasse.java` indica un sorgente. L'estensione `.class` non va aggiunta al nome della classe da avviare. JDK, JRE e JVM descrivono ruoli distinti; la loro distribuzione concreta dipende dal JDK installato.

2.2 Classi, oggetti, campi e accesso allo stato

Il codice di una classe ne definisce struttura, comportamento e relazioni prima che siano create le istanze. Gli oggetti, invece, vengono creati durante l'esecuzione e interagiscono per soddisfare i requisiti del problema. La classe di un oggetto, una volta creato, è fissata: il suo tipo a runtime non cambia.

Un *campo* o *attributo* è una variabile appartenente a una classe o a un oggetto. Un *metodo* è un'operazione invocabile su una classe o su un oggetto. Una variabile di istanza non inizializzata assume valori di default: `0` per i tipi numerici, `false` per `boolean`, `null` per i riferimenti. Una variabile locale, cioè dichiarata dentro un metodo, deve invece essere inizializzata prima dell'uso.

L'*incapsulamento* consiste nel nascondere i dettagli interni di un oggetto e nel permettere l'accesso allo stato attraverso metodi appropriati. La visibilità controlla quali parti del programma possono accedere a campi e metodi:

`private` visibile solo all'interno della stessa classe;

`public` visibile a tutti i client che possono accedere alla classe;

`protected` visibile alla classe, alle classi dello stesso package e, con le regole di accesso previste da Java, alle sottoclassi.

Un oggetto ben progettato espone operazioni coerenti con la propria responsabilità e protegge lo stato da modifiche incompatibili con il contratto. Il *costruttore* inizializza una nuova istanza; l'espressione `new` richiede la creazione dell'oggetto e ne restituisce un riferimento. Un riferimento consente di raggiungere un oggetto e va distinto dai dati che l'oggetto contiene.

Senza modificatore si ha accesso *package-private*, limitato alle classi dello stesso package. `protected` comprende sia l'accesso nel package sia quello previsto per le sottoclassi; non significa soltanto "visibile alle sottoclassi".

2.2.1 Un programma completo: membri statici e membri di istanza

`String` rappresenta testo; `LocalDate`, importata da `java.time`, una data senza ora. `LocalDate.now()` legge la data corrente; `System.out` è il flusso di uscita standard e `println` stampa aggiungendo un fine riga.

```
import java.time.LocalDate;

public class HelloWorld {
    private static final String msg = "Lezione di Java";
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);
        System.out.println(LocalDate.now());
    }
}
```

Salvando il testo in `HelloWorld.java`, `javac HelloWorld.java` lo compila e `java HelloWorld` lo avvia. Le tre righe sono `Hello World`, `Lezione di Java` e la data dell'esecuzione: per esempio `2025-03-03` se eseguito quel giorno. Il metodo statico usa direttamente il campo statico `msg`; per conservare una data distinta per ciascun oggetto si può invece scrivere:

```
import java.time.LocalDate;

public class HelloWorld {
    private final LocalDate d = LocalDate.now();
    public void printDate() { System.out.println(d); }
    public static void main(String[] args) {
        HelloWorld world = new HelloWorld();
        System.out.println("Hello World");
        System.out.println("Lezione di Java");
        world.printDate();
    }
}
```

Le due classi sono alternative da compilare separatamente. Nella seconda il diagramma di classe contiene il campo privato `d: LocalDate`, l'operazione pubblica `printDate(): void` e `main` statico. `main` crea l'istanza `world`; `printDate` legge la data conservata nell'istanza, che non viene ricalcolata a ogni chiamata.

Le parole chiave precisano il contratto: `class` dichiara una classe; `new` crea un oggetto e ne invoca il costruttore; `import` permette di usare un nome di tipo senza scriverne ogni volta il package; `public` rende un membro accessibile ai client, mentre `private` ne limita l'accesso alla classe. `static` associa il membro alla classe; `void` indica che il metodo non restituisce un valore. `return espressione;` restituisce un valore e conclude il metodo; `return;` conclude anticipatamente un metodo `void`.

`final` ha tre usi da distinguere. Una variabile o un campo finale può essere assegnato una sola volta; una classe finale non può essere estesa; un metodo finale non può essere ridefinito. Un riferimento finale non rende immutabile l'oggetto: una `final List<String>` può ancora ricevere elementi, ma la variabile non può essere riassegnata a un'altra lista.

2.3 Package, collection, tipi generici e iterazione

Un *package* è uno spazio di nomi che organizza le classi Java. Le classi del package `java.lang`, come `String`, `System` e `Math`, sono importate automaticamente. Per usare classi di altri

package occorre un'istruzione `import`; ad esempio `import java.time.*` rende disponibili le classi del package indicato. L'import rende un nome disponibile al compilatore e non copia nel bytecode tutte le classi del package; inoltre, l'asterisco non importa i sottopackage.

Le *collection* sono interfacce e classi della JRE, nel package `java.util`, per gestire gruppi di oggetti. Le principali interfacce sono:

`List` sequenza ordinata, ammette duplicati e accesso per indice;

`Set` insieme senza duplicati; l'ordine non è garantito in generale;

`Map` struttura a coppie chiave-valore, con chiavi uniche.

`List` e `Set` estendono `Collection`; `Map` non estende `Collection`, perché raggruppa coppie chiave-valore anziché singoli elementi. `Collection` estende `Iterable`; un oggetto `Iterable` sa produrre un *iteratore*, cioè un oggetto che consente di scorrere una sequenza tramite `hasNext()` e `next()`.

Esempi di implementazioni alternative sono `ArrayList`/`LinkedList`, `HashSet`/`TreeSet` e `HashMap`/`TreeMap`. La scelta dell'interfaccia stabilisce le operazioni disponibili, mentre quella della concreta determina rappresentazione, costi e garanzie ulteriori: per esempio TreeSet e TreeMap mantengono un ordinamento, a differenza di HashSet e HashMap che non garantiscono un ordine di iterazione.

Il ciclo *enhanced for* scorre tutti gli elementi senza gestire esplicitamente gli indici:

```
for (float v: valori) totale += v;
```

L'enhanced for è una forma di *zucchero sintattico*: rende più concisa l'iterazione. Per le istanze di `Iterable`, il compilatore usa `iterator()`, `hasNext()` e `next()`; il ciclo è disponibile anche sugli array, che non devono implementare `Iterable`.

2.3.1 Generics, boxing e tipi primitivi

I *generics* parametrizzano un tipo con altri tipi: `List<T>` descrive una lista di elementi `T`, e `List<Float>` una lista di riferimenti a `Float`. Il compilatore può così controllare il tipo degli elementi. I parametri devono essere tipi di riferimento: `List<Integer>` è valido, `List<int>` non lo è. `int` rappresenta un valore primitivo; `Integer` è la classe che lo incapsula in un oggetto. L'*autoboxing* converte il valore primitivo nel corrispondente oggetto, mentre l'*unboxing* effettua la conversione inversa. Il valore numerico e l'oggetto che lo rappresenta restano concetti distinti.

2.3.2 Enhanced for e Iterator

Implementare `Iterable` permette anche a una collezione personalizzata di essere percorsa con l'enhanced for. Il client usa il protocollo dell'iteratore senza conoscere se gli elementi siano conservati in un array, in una lista o in un'altra struttura.

2.4 Interfacce, classi astratte e contratti

2.4.1 Programmare per le interfacce

Un'*interfaccia* Java è un tipo che definisce un contratto: stabilisce che cosa un oggetto sa fare, senza specificare come lo fa. Dichiara i metodi specificando nome, parametri e tipo di ritorno. La *firma* Java identifica il metodo attraverso nome e tipi dei parametri; il tipo di ritorno appartiene alla dichiarazione, ma non distingue da solo due metodi. Non può avere costruttori né attributi di istanza non inizializzati. Se in un'interfaccia si dichiara un attributo, il compilatore lo tratta come costante `public static final`. Sono disponibili anche i metodi

`default`, dotati di corpo: da Java 8 consentono di aggiungere un'operazione a un'interfaccia esistente senza obbligare immediatamente tutte le implementazioni precedenti a definirla. Un'interfaccia con un unico metodo *astratto* è detta funzionale; eventuali metodi default non aumentano il numero di metodi astratti da implementare.

Una *implementazione* è una classe che fornisce il corpo dei metodi richiesti dall'interfaccia. Per esempio, `List` definisce operazioni come `add()`, `get()`, `size()`, `remove()` e `contains()`; `ArrayList` e `LinkedList` sono implementazioni diverse.

Il principio di progettazione è: *programmare per le interfacce, non per le implementazioni*. Ciò significa dichiarare variabili, parametri e tipi di ritorno con il tipo più astratto utile. Per esempio:

```
List<Float> valori = new ArrayList<>();
```

La variabile `valori` è dichiarata come `List<Float>`, mentre l'oggetto creato è un `ArrayList`. Se in futuro si scegliesse `LinkedList`, il codice client che dipende solo da `List` potrebbe rimanere invariato. Questo riduce la propagazione dei cambiamenti e facilita i test, perché un'implementazione reale può essere sostituita da una fittizia quando si vuole isolare una classe.

2.4.2 Classi astratte e differenze rispetto alle interfacce

Una *classe astratta* non può essere istanziata direttamente. Può contenere metodi concreti, cioè con corpo, e metodi `abstract`, cioè senza implementazione. Un metodo astratto è utile perché altri metodi già implementati possono invocarlo, i client possono aspettarsi che l'operazione esista, e le sottoclassi concrete sono obbligate a implementarlo. Se una sottoclasse non implementa tutti i metodi astratti ereditati, deve essere a sua volta dichiarata astratta.

La scelta dipende dalla relazione da esprimere: un'interfaccia definisce capacità condivisibili anche da classi non correlate; una classe astratta si usa quando esiste una relazione autentica fra classi, del tipo "è un", e quando alcune parti del comportamento sono comuni. Inoltre una classe può implementare più interfacce, ma in Java può estendere una sola classe, anche astratta. L'ereditarietà multipla di implementazione può creare il *diamond problem*: se una classe ereditasse da due superclassi con lo stesso metodo, sarebbe ambiguo quale implementazione usare.

2.4.3 Contratto e sostituzione: IAccount e AccountV2

```
interface IAccount {
    void setBalance(float value);
}
class AccountV2 implements IAccount {
    private float balance;
    @Override public void setBalance(float value) { balance = value; }
    public float getBalance() { return balance; }
}
```

Un client che riceve `IAccount acc` può invocare `acc.setBalance(1234f)` senza conoscere la rappresentazione del saldo. Il punto di configurazione può fornire `new AccountV2()` o un'altra implementazione compatibile. Il getter, presente solo nella concreta, non diventa automaticamente parte del contratto `IAccount`. I metodi astratti dell'interfaccia sono pubblici anche quando la parola `public` è omessa.

Un metodo `default` non dispone di campi di istanza propri dell'interfaccia, ma può chiamare operazioni del contratto che modificano lo stato dell'implementazione: non implica quindi, in generale, assenza di effetti.

2.4.4 Stato comune e operazione astratta: Libro

```
abstract class Libro {
    private final String autore;
    protected final java.util.List<Pagina> pagine =
        new java.util.ArrayList<>();
    protected Libro(String autore) { this.autore = autore; }
    public String autore() { return autore; }
    public abstract void inserisci(Pagina p);
}
record Pagina(String testo) { }
```

`Libro` conserva autore e pagine, ma rinvia a una sottoclasse concreta la politica di inserimento. `Pagina` è qui un semplice valore testuale. Non si può eseguire `new Libro(...)`; una sottoclasse deve fornire `inserisci`. Lo stesso criterio distingue un `Animale` astratto, che può implementare `respira()` e lasciare astratto `verso()`, da `Cane` e `Gatto`, che specializzano il verso. Un'interfaccia è invece adatta a una capacità comune senza imporre quella gerarchia di stato e implementazione.

2.5 Ereditarietà, specializzazione e riuso

L'*ereditarietà* permette di definire una nuova classe indicando ciò che ha in più o di diverso rispetto a una classe esistente. La classe esistente è detta *superclasse*; la nuova classe è detta *sottoclasse*. In Java si usa la parola chiave `extends`:

```
public class Studente extends Persona {... }
```

Una sottoclasse:

1. eredita metodi e attributi della superclasse e può usarli come se fossero definiti localmente, nel rispetto della visibilità;
2. aggiunge nuovi attributi e metodi;
3. può ridefinire, cioè fare *override*, dei metodi della superclasse;
4. non può eliminare metodi o attributi della superclasse.

La parola chiave `super` permette a una sottoclasse di richiamare un metodo o un costruttore della superclasse. L'annotazione `@Override` comunica al compilatore che il metodo dovrebbe ridefinire un metodo ereditato: se la signature non corrisponde, il compilatore segnala errore, proteggendo da sbagli di nome o parametri.

Il riuso tramite ereditarietà è vantaggioso perché evita di copiare classi esistenti e modificarle, con proliferazione di codice e lavoro manuale. Tuttavia richiede comprensione della superclasse e può aumentare il legame fra superclasse e sottoclasse; per questo, quando non esiste una relazione "è un", può essere preferibile la composizione, cioè costruire una classe che contiene e usa oggetti di altre classi invece di ereditarne l'implementazione.

Il nome della sottoclasse dovrebbe comunicare sia la somiglianza sia la specializzazione rispetto alla base: i nomi alle radici della gerarchia sono solitamente più generali e concisi, quelli delle specializzazioni più specifici. L'ereditarietà riusa l'implementazione senza copiarla e senza richiedere di modificare la classe già funzionante.

2.5.1 Esempio Persona, Studente ed Esame

Si consideri `Persona`, con campi `nome` e `cognome` e metodo `printAll()`. `Studente` ne è una specializzazione: aggiunge `matricola`, un elenco di esami e operazioni `nuovoEsame()` e

`getMedia()`; usa `super(nome, cognome)` nel costruttore e ridefinisce `printAll()` per mostrare anche i dati aggiuntivi. Ogni `Esame` raggruppa `materia` e `voto`; può essere rappresentato mediante un *record*, una classe dedicata a un insieme di componenti di dati. L'annotazione `@Override` permette al compilatore di verificare la compatibilità della ridefinizione.

```
import java.util.ArrayList;
import java.util.List;

class Persona {
    private final String nome;
    private final String cognome;
    Persona(String nome, String cognome) {
        this.nome = nome;
        this.cognome = cognome;
    }
    public void printAll() {
        System.out.println(nome + " " + cognome);
    }
}
record Esame(String materia, int voto) { }
class Studente extends Persona {
    private final String matricola;
    private final List<Esame> esami = new ArrayList<>();
    Studente(String nome, String cognome, String matricola) {
        super(nome, cognome);
        this.matricola = matricola;
    }
    public void nuovoEsame(String materia, int voto) {
        esami.add(new Esame(materia, voto));
    }
    public float getMedia() {
        if (esami.isEmpty()) return 0;
        float somma = 0;
        for (Esame e : esami) somma += e.voto();
        return somma / esami.size();
    }
    @Override public void printAll() {
        super.printAll();
        System.out.println(matricola);
        for (Esame e : esami)
            System.out.println(e.materia() + " " + e.voto());
        System.out.println("Media: " + getMedia());
    }
}
```

Il costruttore di `Studente` inizializza la parte `Persona` mediante `super`; gli esami sono oggetti distinti aggiunti alla lista. Il tipo record espone `materia()` e `voto()`, non getter con prefisso `get`. La media è zero per lista vuota secondo il contratto scelto; per due esami con voti 8 e 7 è 7,5.

L'esecuzione seguente mostra compatibilità fra tipi e late binding:

```
Studente s = new Studente("Alan", "Rossi", "M12345");
s.nuovoEsame("Italiano", 8);
s.nuovoEsame("Fisica", 7);
s.printAll();
Persona p = s;
p.printAll();
```

La variabile `p` è dichiarata `Persona`, ma contiene un riferimento a un oggetto `Studente`; la chiamata a `printAll()` esegue quindi la versione di `Studente`. Non sarebbe invece ammesso chiamare `p.nuovoEsame(...)` senza cast, perché `nuovoEsame` non appartiene al tipo dichiarato `Persona`.

2.6 Compatibilità dei tipi, cast e uguaglianza

L'ereditarietà definisce una classificazione di tipi. Una sottoclasse è un *sottotipo* compatibile con la superclasse: ogni `Studente` è anche una `Persona`, ma non ogni `Persona` è uno `Studente`. Per questo è lecito assegnare un riferimento a `Studente` a una variabile dichiarata `Persona`:

```
Studente s = new Studente("Alan", "Rossi", "M12345");
Persona p = s;
p.printAll();
```

A tempo di compilazione, il tipo dichiarato della variabile determina quali operazioni possono essere invocate. Su `p` si può chiamare `printAll()` se tale metodo esiste in `Persona`; non si può chiamare `nuovoEsame()`, anche se a runtime `p` punta a uno `Studente`, perché il compilatore conosce `p` come `Persona`.

Il *cast* esplicito forza il compilatore a trattare un riferimento come un sottotipo:

```
Persona p = new Studente("Alan", "Rossi", "M12345");
Studente s = (Studente) p;
```

Il cast è accettato dal compilatore quando è plausibile nella gerarchia, ma a runtime riesce solo se l'oggetto è davvero del tipo richiesto. In caso contrario viene generata `ClassCastException`. Per esempio, un cast da `new Persona(...)` a `Studente` fallirebbe a runtime.

2.6.1 Identità degli oggetti e confronto del contenuto

Per gli oggetti, `==` confronta i riferimenti; `equals()` può confrontare il contenuto secondo il contratto del tipo. Due stringhe con lo stesso contenuto possono essere oggetti diversi: il confronto corretto del testo usa `equals()`, eventualmente `Objects.equals(a,b)` se si vuole gestire `null`. La lunghezza della stringa non determina la semantica di `==`; il fatto che alcuni letterali condividano l'istanza spiega perché esempi semplici possano ingannare.

2.7 Polimorfismo, binding e dispatch

Il *polimorfismo* è la proprietà per cui la stessa invocazione di metodo può produrre comportamenti diversi in base al tipo concreto dell'oggetto che la riceve. Nei sistemi a oggetti possono esistere metodi con stesso nome e stessa signature in classi diverse della gerarchia; se una sottoclasse ridefinisce un metodo, la chiamata viene risolta dinamicamente.

Il *late binding* è il collegamento dinamico: a runtime si guarda il tipo effettivo dell'istanza, non soltanto il tipo dichiarato della variabile. Il *early binding*, al contrario, collegherebbe staticamente la chiamata in base al tipo dichiarato. In Java il dispatch dei metodi di istanza ridefinibili è dinamico.

Il *dispatch* è il meccanismo con cui Java decide quale versione di un metodo chiamare. Per un metodo di istanza ridefinibile, il meccanismo si può descrivere così:

1. quando un metodo viene chiamato su un oggetto, si considera il tipo dell'istanza a runtime;
2. si cerca il metodo nella classe concreta dell'istanza;

3. se non viene trovato, si risale la gerarchia delle superclassi finché il metodo viene trovato;
4. il metodo trovato viene eseguito.

Nell'esempio `Account`/`SavingAccount`, una variabile `Account acc = new SavingAccount()` riceve la chiamata `acc.setBalance(1234)`. Il metodo `setBalance()` viene trovato in `Account`; dentro tale metodo viene chiamato `check()`. Poiché l'oggetto reale è `SavingAccount` e `SavingAccount` ridefinisce `check()`, viene eseguito `SavingAccount.check()`. L'esempio mostra che anche da un metodo della superclasse una chiamata a un metodo ridefinito può essere risolta sulla sottoclasse.

Senza polimorfismo, il chiamante dovrebbe contenere uno `switch` o una sequenza di condizioni per distinguere manualmente le classi degli oggetti e chiamare il metodo adatto. Ciò rende il codice più fragile e difficile da manutenere.

Un client può scegliere l'oggetto con una condizione e invocare poi un unico contratto:

```
Persona p = new Persona("Alan", "Rossi");
Studente s = new Studente("Alan", "Rossi", "M12345");
Persona px;
if (i > 10) px = p;
else px = s;
px.printAll();
```

Il valore di `i` decide il riferimento; il tipo dinamico decide l'override. Un terzo livello `StudenteLavor extends Studente` può ridefinire `nuovoEsame` e `printAll`: la ricerca parte da StudenteLavor, risale a Studente per i metodi non ridefiniti e poi a Persona. Non si assume una specifica regola lavorativa non definita dal contratto.

2.7.1 Overload e override

L'*overload* permette di dichiarare più metodi con lo stesso nome e parametri differenti. Non basta cambiare il tipo di ritorno. La scelta dell'overload avviene in compilazione in base ai tipi disponibili; il dispatch dinamico sceglie poi l'*override*, cioè la ridefinizione del metodo di istanza selezionato. Campi, metodi statici e costruttori non seguono questo meccanismo di override dinamico.

2.7.2 Account e SavingAccount: chiamata dalla superclasse alla sottoclasse

```java
public class Account {
   protected float balance = 0;
   public void setBalance(float amount) {
      System.out.println("in Account set-balance");
      if (check(amount))
         balance += amount;
   }
   public boolean check(float amount) {
      System.out.println("in Account check");
      return balance + amount >= 0;
   }
}

public class SavingAccount extends Account {
   public boolean check(float amount) {
      System.out.println("in Saving-Account check");
      return (balance + amount) >= 1000;
   }
}

public class AccTest {
   public static void main(String[] args) {
      Account acc = new SavingAccount();
      acc.setBalance(1234);
   }
}
```

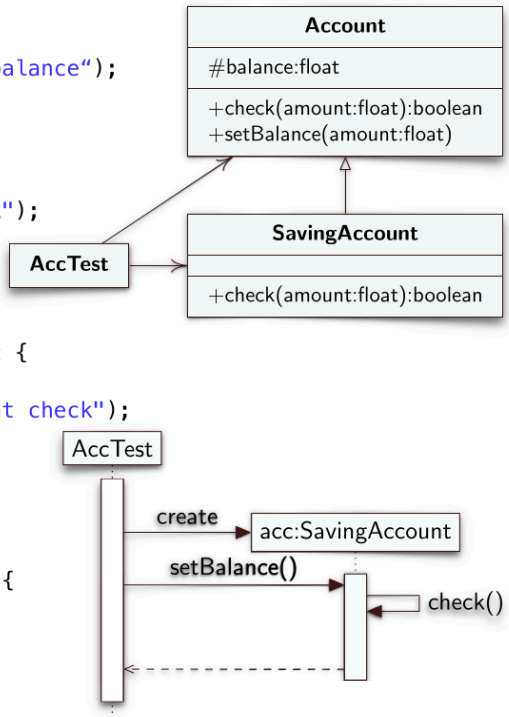


Figura 2.1: Codice e diagrammi del dispatch fra `Account` e `SavingAccount`.

Nell'esempio, `setBalance()` è ereditato da `Account`, mentre `check()` è ridefinito in `SavingAccount`. La chiamata interna a `check()` usa ancora il ricevente reale, cioè `this`: non diventa una chiamata su un altro oggetto della superclasse. È questo il motivo per cui un algoritmo ereditato può usare un passo specializzato, meccanismo che riappare nel Template Method.

2.8 Record e immutabilità dei dati

Un *record* Java è una classe speciale progettata per contenere dati immutabili. Per esempio:

```
public record Esame(String materia, int voto) { }
```

Il record fornisce automaticamente un costruttore che assegna tutti i campi e metodi di accesso con lo stesso nome del campo, per esempio `materia()` e `voto()`, non `getMateria()`. I campi sono `final`, quindi non modificabili dopo l'assegnazione, e non sono presenti setter. I record sono `final`: non possono essere estesi da sottoclassi. Possono implementare interfacce, ma non estendere esplicitamente altre classi; possono inoltre avere campi statici, metodi statici e metodi di istanza aggiuntivi.

Nell'esempio `Studente`, `Esame` è usato come contenitore immutabile di materia e voto. La media dei voti può essere descritta dalla formula:

$$\bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i, \tag{2.2}$$

dove n è il numero di esami presenti nella lista, v_i è il voto dell'i-esimo esame e $\bar{v}$ è la media. Se si sceglie di rappresentare il caso senza esami con il valore zero, `getMedia()` restituisce `0`; quindi la formula si applica solo per $n > 0$.

I campi finali di un record impediscono di riassegnare i riferimenti, ma non rendono automaticamente immutabili gli oggetti raggiungibili. Se un componente è una lista mutabile, il suo contenuto può ancora cambiare; per un valore davvero protetto occorre scegliere componenti immutabili o copie difensive. Oltre a costruttore e accessori, un record dispone di implementazioni di `equals`, `hashCode` e `toString` coerenti con i componenti.

2.9 Inferenza del tipo delle variabili locali

`var` permette al compilatore di inferire il tipo di una variabile locale dall'inizializzatore; non introduce tipizzazione dinamica. È disponibile da Java 10. Per esempio:

```
var valori = new ArrayList<String>();
// Il tipo statico inferito e' ArrayList<String>.
List<String> contratto = valori;
```

Non si usa `var` per un campo, per dichiarare una variabile senza inizializzarla o con il solo `null`. Inoltre il tipo inferito può essere più concreto dell'interfaccia che si voleva esporre: scegliere `List<String>` resta utile quando rende esplicito il contratto.

2.10 Caso progressivo: leggere e aggregare importi

Un'applicazione deve leggere più file di testo, ciascuno con un numero per riga, eliminare i duplicati e calcolare somma e massimo dei valori distinti. I quattro requisiti vanno controllati separatamente: leggere correttamente un file non dimostra né la deduplicazione né il corretto calcolo degli aggregati.

2.10.1 Dalla lettura monolitica alla separazione dei compiti

La prima versione di `CalcolaImporti` legge e somma nello stesso ciclo:

```
import java.io.*;
import java.util.*;

class CalcolaImporti {
    private final List<String> importi = new ArrayList<>();
    public float calcola(String cartella, String nome) {
        float totale = 0;
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(cartella, nome)))) {
            String riga;
            while ((riga = f.readLine()) != null) {
                try {
                    float valore = Float.parseFloat(riga);
                    importi.add(riga);
                    totale += valore;
                } catch (NumberFormatException e) {
                    System.err.println("Numero non valido: " + riga);
                }
            }
        } catch (IOException e) {
            System.err.println("Errore di I/O: " + e.getMessage());
        }
        return totale;
    }
}
```

`float` è un tipo numerico primitivo in virgola mobile. `while` ripete il corpo finché la condizione è vera; `readLine()` legge una riga senza il terminatore e restituisce `null` a fine file. L'assegnazione nella condizione conserva la riga prima del controllo. `Float.parseFloat` converte il testo e può sollevare `NumberFormatException`; un problema di apertura o lettura è invece un'`IOException`. I blocchi `try/catch` distinguono i due eventi e il *try-with-resources* chiude il lettore anche in caso di errore. `if` esegue un blocco soltanto quando la propria condizione è vera.

Questa versione realizza lettura e somma, ma conta anche i duplicati e non calcola il massimo. Aggiungere `if (valore > massimo) massimo = valore;` realizza il massimo solo con una corretta inizializzazione: partire da zero è adeguato per importi non negativi, mentre sbaglia un insieme composto da valori negativi. La versione successiva aggiunge prima dell'inserimento `if (!importi.contains(riga))`: il ciclo aggiorna somma e massimo soltanto per righe nuove. Occorre però precisare che `"1"` e `"1.0"` sono testi diversi pur rappresentando lo stesso numero.

`List.add` aggiunge un elemento; `contains` verifica l'uguaglianza degli elementi secondo `equals`, non la loro sola identità. Per una lista di stringhe deduplica quindi il testo; per una lista di `Float` usa il contratto di uguaglianza del tipo numerico. La rappresentazione scelta influenza il significato di "duplicato".

Un'ulteriore versione legge prima tutte le righe distinte e poi svolge due cicli:

```
float totale = 0;
for (String s : importi) totale += Float.parseFloat(s);
float massimo = Float.parseFloat(importi.get(0));
for (String s : importi) {
    float valore = Float.parseFloat(s);
    if (valore > massimo) massimo = valore;
}
```

L'enhanced for assegna a `s` ogni elemento della lista. Il massimo parte dal primo elemento e gestisce anche numeri negativi, ma richiede una lista non vuota. I due cicli separano i calcoli, pur ripetendo le conversioni. La lunghezza può crescere anche se l'intento diventa più leggibile: le versioni con un unico metodo hanno rispettivamente 18 e 23 linee nella convenzione di conteggio a punto e virgola.

Mescolare tutto in un metodo rende difficile osservare ciò che è stato letto, testare separatamente somma e massimo, cambiare la regola sui duplicati o riusare il calcolo senza file. Inesperienza e assenza di progettazione favoriscono questa struttura, soprattutto quando si aggiungono requisiti senza riesaminare responsabilità e dipendenze. Una classe chiamata come una procedura non diventa un buon modello a oggetti per il solo fatto di essere una classe.

2.10.2 Pagamenti con comandi di calcolo e query di lettura

La separazione produce `Pagamenti`: `leggiFile` invoca `inserisci`; `converti` trasforma le righe in numeri; `calcolaSomma` e `calcolaMassimo` aggiornano campi derivati; `getSomma` e `getMassimo` li osservano. I campi sono `importiLetti: List<String>`, `valori: List<Float>`, `totale` e `massimo`.

```
class PagamentiPreparati {
    private final List<String> importiLetti = new ArrayList<>();
    private final List<Float> valori = new ArrayList<>();
    private float totale, massimo;
    public void inserisci(String s) {
        if (!importiLetti.contains(s)) importiLetti.add(s);
    }
    public void leggiFile(String c, String n) throws IOException {
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(c, n)))) {
```

```
            String riga;
            while ((riga = f.readLine()) != null) inserisci(riga);
        }
    }
    public void converti() {
        valori.clear();
        for (String s : importiLetti) {
            try { valori.add(Float.parseFloat(s)); }
            catch (NumberFormatException e) {
                System.err.println("Numero non valido: " + s);
            }
        }
    }
    public void calcolaSomma() {
        totale = 0;
        for (float v : valori) totale += v;
    }
    public void calcolaMassimo() {
        massimo = 0;
        if (valori.isEmpty()) return;
        massimo = valori.get(0);
        for (float v : valori) if (v > massimo) massimo = v;
    }
    public float getSomma() { return totale; }
    public float getMassimo() { return massimo; }
    public void svuota() {
        importiLetti.clear(); valori.clear();
        totale = 0; massimo = 0;
    }
}
```

Gli import sono quelli del listato precedente. Il nome `PagamentiPreparati` distingue questa variante dalla successiva. Svuotare `valori` prima di convertire impedisce che due conversioni consecutive accumulino copie degli stessi dati. Gli errori di I/O sono qui propagati al client, che decide come presentarli; gli errori di conversione sono segnalati scartando la riga. Queste sono politiche distinte e fanno parte del contratto.

La sequenza del client è `svuota`, una o più `leggiFile`, `converti`, i due comandi di calcolo e infine le query. Se si aggiungono dati dopo il calcolo, i risultati memorizzati diventano obsoleti fino alla nuova esecuzione dei comandi. L'ordine è dunque un vincolo osservabile, non un dettaglio interno.

2.10.3 Pagamenti con risultati calcolati su richiesta

Eliminare righe testuali e campi derivati riduce lo stato da mantenere coerente. Il contratto di questa variante deduplica valori `Float`, calcola gli aggregati quando richiesti e restituisce convenzionalmente zero sulla lista vuota:

```
class Pagamenti {
    private final List<Float> importi = new ArrayList<>();
    public void leggiFile(String c, String n) throws IOException {
        try (BufferedReader f = new BufferedReader(
                new FileReader(new File(c, n)))) {
            String riga;
            while ((riga = f.readLine()) != null)
                inserisci(Float.parseFloat(riga));
        }
```

```
    }
    public void inserisci(float x) {
        if (!importi.contains(x)) importi.add(x);
    }
    public float calcolaSomma() {
        float risultato = 0;
        for (float v : importi) risultato += v;
        return risultato;
    }
    public float calcolaMassimo() {
        if (importi.isEmpty()) return 0;
        float risultato = importi.get(0);
        for (float v : importi)
            if (v > risultato) risultato = v;
        return risultato;
    }
    public void svuota() { importi.clear(); }
}
```

`calcolaSomma` e `calcolaMassimo` sono ora query: restituiscono il risultato senza aggiornare lo stato. La CQS continua a essere rispettata, anche senza una coppia separata comando/getter. `IOException` e `NumberFormatException` raggiungono il chiamante; una lettura fallita dopo alcune righe può aver già inserito una parte dei dati, perché l'operazione non è transazionale. Inizializzare il massimo al primo elemento conserva il significato anche per importi negativi; l'alternativa con accumulatore iniziale zero richiede esplicitamente un dominio non negativo. Le due soluzioni coincidono solo in quel dominio.

2.10.4 Una raccolta comune o una per file?

```
Pagamenti insieme = new Pagamenti();
insieme.leggiFile("./", "Importi.csv");
System.out.println(insieme.calcolaSomma());
insieme.leggiFile("./", "AltriImporti.csv");
System.out.println(insieme.calcolaSomma());

Pagamenti primo = new Pagamenti();
Pagamenti secondo = new Pagamenti();
primo.leggiFile("./", "Importi.csv");
secondo.leggiFile("./", "AltriImporti.csv");
System.out.println(primo.calcolaSomma());
System.out.println(secondo.calcolaSomma());
```

Nel primo caso la seconda lettura aggiunge dati alla raccolta esistente e deduplica anche fra file; nel secondo le raccolte sono indipendenti. Il client deve gestire o dichiarare `IOException`. Per ottenere risultati separati riusando la stessa istanza occorre `svuota()` fra i file. La scelta del numero di oggetti deriva dal requisito: una sola classe non implica una sola istanza e non giustifica da sola Singleton.

Per controllare la comprensione del caso, si devono saper spiegare il ciclo enhanced for, la differenza fra `List` e `ArrayList`, il significato di `contains`, quali operazioni sono query e come verificare la correttezza. Le risposte si collegano: l'interfaccia permette di cambiare contenitore, l'uguaglianza determina i duplicati e le query offrono risultati osservabili da confrontare con un oracle.

2.11 Progettazione a oggetti: procedimento operativo

Per attribuire responsabilità e definire collaborazioni si può procedere in questo modo:

1. individuare le entità del dominio e dare loro nomi nominali, per esempio `Persona`, `Studente`, `Esame`;
2. attribuire a ciascuna classe uno stato coerente e responsabilità limitate;
3. nascondere i dettagli interni con visibilità adeguata, in particolare usando campi `private` quando lo stato non deve essere manipolato direttamente dai client;
4. esporre operazioni pubbliche significative, evitando classi usate solo come contenitori di procedure;
5. usare interfacce quando si vuole dipendere da un contratto e non da una classe concreta;
6. usare ereditarietà solo quando la sottoclasse è davvero un caso particolare della superclasse;
7. sfruttare override, late binding e dispatch per lasciare che il comportamento specifico sia scelto dall'oggetto concreto;
8. quando una struttura cresce o accumula responsabilità, migliorarne la struttura con piccoli refactoring protetti dai test.

La scelta di una struttura è motivata dalle responsabilità e dai cambiamenti attesi: un contratto stabile e un numero contenuto di dipendenze rendono più semplice verificare e modificare il programma.